Multimedia Tools and Applications (2022) 81:27949–27968
https://doi.org/10.1007/s11042-022-12966-1

Particle swarm optimization performance improvement using deep learning techniques

Y.V.R. Naga Pawan[1] · Kolla Bhanu Prakash[2] · Subrata Chowdhury[3] · Yu-Chen Hu[4]

Received: 5 July 2021 / Revised: 28 February 2022 / Accepted: 13 March 2022 /
Published online: 29 March 2022

Abstract

Deep learning is widely used to automate processes, improve performance, detect patterns, and solve problems. Thus, applications of deep learning are limitless. Particle swarm optimization is a computational method that optimizes a problem by trying to improve a candidate solution. Although many researchers proposed particle swarm optimization variants, each variant is unique and superior to the existing ones. Among them, inertia weight-based particle swarm optimization has its own identity. By adjusting the inertia weight, the performance of the swarm can be improved. This paper proposes two new particle swarm optimization models using the convolutional neural network and long short-term memory to predict the inertia weight in moving the swarm for improving the swarm performance. The performance of the two new inertia weight models is compared in terms of mean absolute error and standard deviation, with the existing inertia weight based particle swarm optimizations like constant inertia weight, random inertia weight, and linearly decreasing inertia weight particle swarm optimizations.

✉ Yu-Chen Hu
ychu@pu.edu.tw

Y.V.R. Naga Pawan
ynpawan@gmail.com

Kolla Bhanu Prakash
drkbp@kluniversity.in

Subrata Chowdhury
subrata895@gmail.com

[1] Department of Computer Science & Engineering, Anurag Engineering College, Ananthagiri (V&M), Telangana, India

[2] Department of Computer Science & Engineering, Koneru Lakshmaiah Education Foundation, Vaddeswaram, Andhra Pradesh, India

[3] Department of Computer Science & Applications, SVCET Engineering College, Chittoor, Andhra Pradesh, India

[4] Department of Computer Science & Information Management, Providence University, 200, Sec. 7, Taiwan Boulevard, Shalu Dist, Taichung City 43301 Taiwan, Republic of China

Experiments are conducted with swarm sizes 50, 75, and 100 with dimensions 10, 15, and 25 using the five most commonly used benchmark functions. The results show that the new models have significant performance gain over existing constant, random and linearly decreasing inertia weight particle swarm optimization models.

Keywords Inertia weight · Convolutional neural network · Long short-term memory · Particle swarm optimization

1 Introduction

Reynolds developed an artificial life program Boids. It simulates the flocking behavior of the birds [26]. Next, Kennedy and Eberhart proposed Particle Swarm Optimization (PSO) [13]. It is a population-based stochastic optimization technique inspired by the movement of bird flocks and fish schools inspired by the Reynolds Boid Model [5]. PSO has many similarities with evolutionary computing technology. The idea of PSO is to initialize the system with many random solutions and find the best solution by updating the generations. Each swarm member is termed as a particle; each particle has a fitness value, a potential solution to the issue of optimization. PSO's primary aim is to identify the ideal or sub ideal solution of an objective function via social interaction and the sharing of knowledge amongst the particles [15]. The simplicity and fast convergence of the PSO it is applied in many gamuts of domains to solve complex mathematical and engineering problems [5, 6]. Initially, PSO is applied in solving non-linear continuous functions [20]. PSO is widely used in electrical, electronic, data, computing, mechanical, medicine, networks, chemical, communications, military, security, etc. In addition, PSO is also applied in identification systems, multi-objective optimization, automatic target detection, time-frequency analysis, multimedia, and so on [20].

Numerous variants are developed to improve the performance of PSO based on initialization, compression factor, Inertia Weight (IW), mutation operator, niche mechanism, group algorithm, fuzzy logic, parallelism, dynamic hierarchical structure, p and q parameters, neighborhood, multi-swarm, quantum mechanics, etc. [7, 23, 35]. Among existing variants of PSOs, IW-based PSOs have better performance [2].

In recent years, due to the powerful ability of Deep Learning Technology (DLT) to learn advanced features from data, it has gained momentum. It enables computational models to learn features from multiple levels of data gradually. In IWPSO, if a suitable IW is identified, the performance of PSO will be better. If deep learning models predict, a fitting IW, the performance of the IWPSO will be increased.

In this paper, DLT has been integrated into PSO based on Inertia Weight (IWPSO) to improve the performance of PSO. Two new IWPSOs are proposed to estimate the inertia weight of PSO, one is to use a Convolutional Neural Network (CNN) model, and the other is to use Long Short-Term Memory (LSTM) model to predict inertia weight to improve the performance of the IWPSOs. In terms of Mean Absolute Error (MAE), and Standard Deviation (SD), the performance of the new IWPSOs are compared with the commonly used IWPSO, Constant Inertia Weight PSO (CIWPSO), Random Inertia Weight PSO (RIWPSO), and Linearly Decreasing Inertia Weight PSO (LDIWPSO). The IWPSO, which uses the CNN model for predicting IW, is called Convolutional Neural Network Inertia Weight PSO (CNNIWPSO). The IWPSO, which utilizes the LSTM model for predicting IW, is called Long Short Term Memory Inertia Weight PSO (LSTMIWPSO). The experiments are

conducted to evaluate the performance of CIWPSO, RIWPSO, LDIWPSO, CNNIWPSO and LSTMIWPSO in terms of MAE and SD using five benchmark functions (BMF), Akley, Alpine, Rastrigin, Rosenbrock and Sphere. The results are collected for the swarm sizes, 50, 75, and 100 and the dimensions 10, 15 and 25, for each swarm size. The results are compared in terms MAE and SD.

The rest of the paper is structured as Section 2 describes the processing steps of the Basic Particle swarm optimization (BPSO). In this paper, two novel inertia weight based PSOs are proposed using Convolutional Neural Network and Long-Short Term Memory for predicting Inertia Weights. Section 3 discusses CNNIWPSO. Section discusses LSTMIWPSO. Section 5 presents simulation findings and comparisons of the performance of CIWPSO, RIWPSO, LDIWPSO, CNNIWPSO, and LSTMIWPSO. The conclusion and future scope are presented in Section 6.

2 Particle swarm optimization

2.1 Basic particle swarm optimization

The flock of birds in flight is one of the most beautiful tourist places. The herds and other forms of organizations, such as plants and terrestrial animals, are attractive to observe and think about the behavior of the organizational group. It consists of different birds, but the general exercise is fluent. It is simple but complicated visually. It seems that it is randomly arranged. It is magnificent. The sense of intentional and concentrated authority is the most embarrassed. Besides, all evidence indicates that the flock's movement is only the product of each bird that acts on recognizing that area. In the flocking model, bird-like objects, called boids, are used [26]. Position and speed characterize each boid is well known for what happens in its immediate surroundings. Separation, alignment, and cohesion are the three basic steering behaviors exhibited by the boids [26].

PSO does not require specific problem knowledge, such as gradual changes. It can be used when data cannot be accessed in a particular problem or expensive data. In a swarm, the fitness score of each particle is calculated. Using a fitness score, the particle's best position is identified. The position of each particle represents a possible solution to the optimization problem [13]. Later global best position among the particle best position is determined. Makes use of the best global and local locations to find fascinating areas for further exploration and locations where all this information is shared with other particles, thereby helping particles explore the solution space more effectively. It is an iterative optimization method [36].

Many people are working on PSO to improve their ability to solve optimization problems. PSO is used to solve optimization problems ranging from simple to complex. PSO adjusts the trajectory of the particles to find the space of the solution. Kennedy [13] proposed PSO is BPSO. In BPSO, each particle is considered a point in D-dimensional space, adjusting itself and other neighborhoods. Each particle tracks its coordinates in the problem space. It is also related to the best solution (adaptability) that has been obtained so far. This value is called the particle best (pbest). The best overall value (gbest) is another best value recorded by BPSO and is the highest value obtained so far for any particle near the particle. The BPSO concept involves changing each particle's velocity (or acceleration) towards its optimal position at each step. Each particle seeks to change its current position and velocity based on the particle's current position and pbest and the distance between their current position and gbest. The

fitness function measures the performance of the particles. Different problems have different fitness functions. For example, the birds search for food by randomly following a group of members closest to the food. The birds communicate with each other to achieve the best position. This procedure is continued until the food or solution is found [14].

The formulation of BPSO [3, 23, 28, 35] is done based on the objective function given in eq. (1). The objective function quantifies the resulting solution's proximity to the optimal.

$$f(x) : \mathbb{R}^d \rightarrow \mathbb{R}, \tag{1}$$

where S is search space and d number of dimensions in S. S is a subset of $\mathbb{R}^d$, shown in eq. (2) and defined by eq. (3). Equations (4) and (5) depict the global optimization problem.

$$S \subseteq \mathbb{R}^D \tag{2}$$

$$S = \{px_i \mid px_{min} \leq px_i \leq px_{max}\} \tag{3}$$

$$\min_{x \in S} f(x) \tag{4}$$

The objective function, f(x) needs $px_i \in S$ such that:

$$\forall py_i \in S : f(px_i) \leq f(py_i). \tag{5}$$

In the BPSO, a Swarm, SW, consists of n particles represented as SW = $\{P_1, P_2, P_3, \ldots, P_n\}$. Each Particle P_i has a position in the search space represented by $PX_i = \{px_{i1}, px_{i2}, px_{i3}, \ldots, px_{iD}\}$ where S is the search space, D is the number of dimensions in S. Each particle $P_{i \text{ in } S}$ moves with a velocity V_i, represented as $PV_i = \{pv_{i1}, pv_{i2}, pv_{i3}, \ldots, pv_{iD}\}$. Each particle, P_i, keeps its best position, Pb_i, which is expressed as $Pb_i = \{pb_{i1}, pb_{i2}, pb_{i3}, \ldots, pb_{iD}\}$. The best particle is chosen from the population of all particles and represented as $P_g = \{pg_{i1}, pg_{i2}, pg_{i3}, \ldots, pg_{iD}\}$.The eqs. (6) and (7) provide the fundamental equations governing the operation of the BPSO.

$$pv_{id} = pv_{id} + c_1 * \text{random}() * (pb_i - px_{id}) + c_2 * \text{Random}() * (P_g - px_{id}). \tag{6}$$

$$px_{id} = px_{id} + pv_{id}, \tag{7}$$

where c_1 and c_2 are two positive acceleration coefficients that govern the swarm's relative proportion of cognition, and social interactions, random () and Random () are two random functions in the [0,1]. $pv_{id}$is then clamped to a maximum velocity pv_{max}, the parameter is given by the user. The first part of the eq. (6) represents the particle's initial momentum, the second part represents the particle's cognition, and the third part represents particle cooperation [13, 19, 35].

As particles tend to explore the search space hugely, the velocities of the particles are limited to the constant pv_{max} [29, 33]. The particle velocity is adjusted using (8)

$$pv_{id} = \begin{cases} pv_{id} & \text{if} -pv_{max} \leq pv_{id} \leq pv_{max} \\ pv_{max} & \text{if } pv_{id} > pv_{max} \\ -pv_{max} & \text{if } pv_{id} < -pv_{max} \end{cases} \tag{8}$$

If pv_{max} is larger, it causes global exploration, or the smaller values encourage local exploitation. Typically, the pv_{max} value is chosen as an equal part of the search area using eq. (9) [16 – 17], where δ is the velocity clamping factor.

$$pv_{max} = \delta * (px_{max} - px_{min}) \text{ where } \delta \in (0, 1) \tag{9}$$

As the search space, S, is bounded by the interval $[px_{min}, px_{max}]$, the velocity clamping [18] of the particle is in the interval $[-pv_{max}, pv_{max}]$ represented by $[pv_{min}, pv_{max}]$, where $pv_{max} = \delta * (px_{max} - px_{min}) / 2$.

The flowchart for movement of the particles in BPSO is shown in Fig. 1. The flowchart has three parts, the first part is local, the second part is based on the vicinity, and the last part on global. Uniform distribution over the search space is typically used to initialize the swarm [21]. In every iteration of the swarm movement, new velocities and positions are computed. Using new values, the fitness value is calculated. The particle best and swarm best are updated. This computation is continued until the stopping criterion is used. In general, the stopping criterion is allowable fitness error or number of iterations.

2.2 Inertia weight particle swarm optimization (IWPSO)

Shi and Eberhart [30] improved PSO by introducing a new parameter IW to classic PSO, BPSO, to balance local and global search functions. The purpose of IW is to control

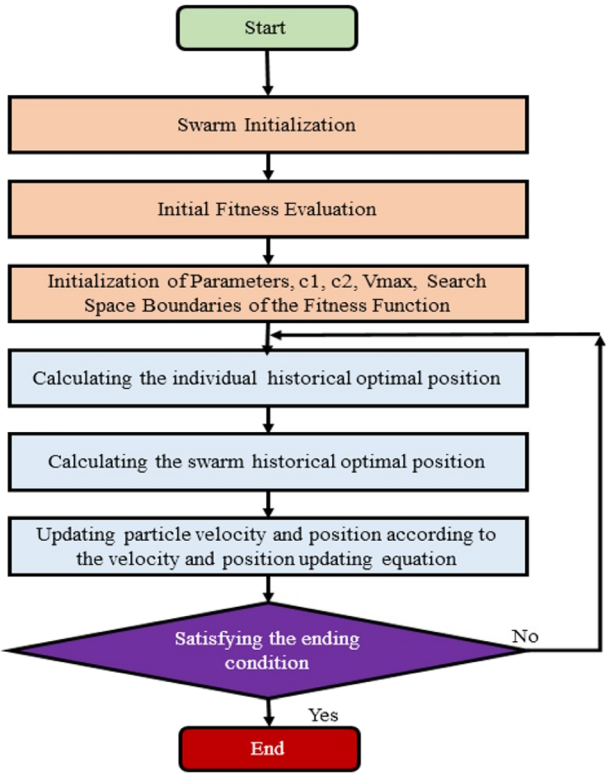


Fig. 1 Flowchart for basic particle swarm optimization

exploration and development better. PSO based on inertia weight is called IWPSO. The result of eq. (6) is shown in eq. (10).

$$pv_{id} = \omega * pv_{id} + c_1 * random\ ()*(pb_i - px_{id}) + c_2 * Random\ ()*(pg_i - px_{id}), \quad (10)$$

where ω is inertia weight.

Bansal et al. [2] studied different IWPSOs. It is seen that only a few IWPSOs have replaced other IWPSOs. Adam [22] observed that the size of the group also affects the performance of the group. Bansal observed that adjusting the IW parameters can improve the performance of IWPSO. Liu [17] utilized IWPSO to enhance the performance of the neural network. Lin and Hu [16] designed the electrical energy management system based on a hybrid artificial neural network-particle swarm optimization-integrated two-stage non-intrusive load monitoring process in smart homes.

3 Proposed CNN-based IWPSO

3.1 Convolutional neural network

The deep learning potential is a large amount of data and has been demonstrated as a beneficial technology [24, 34]. CNN is a class of deep networks and is often used in the recognition of images. It is an artificial neuronal feeding network (ANN) [31]. The pre-processing required by CNN is much lower than other classification algorithms. A typical CNN architecture has three essential layers: convolutional layer, grouping layer, and fully connected layer (dense layer).

In addition to the previous layers, the activation mechanism is another important part of CNN. This function is not as linear as Sigmoid, tanh, Leaky ReLU, ELU, PReLU, and ReLU [11, 12, 25, 31]. The convolution and down-sampling layers or pool layers are often used in series but can be seen in any order. These fully connected layers or linear layers are entirely connected to perform classification or regression tasks.

3.2 CNNIWPSO details

In CNNIWPSO, the new inertia weight is computed using a univariate CNN [10, 12, 27, 32]. Initially, a univariate CNN is built [10, 27]. Then, the CNN model is built and trained with different IWs from 0.05 to 1.00. The sample set of IW with interval 0.05 is given as iw = [0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 1.0]. The training and testing set are shown in Table 1. By reducing the interval, the length of IW set be

Table 1 Sample training and testing sets of IW

Training Set			Test Set
0.05	0.1	0.15	0.2
0.25	0.3	0.35	0.4
0.45	0.5	0.55	0.6
0.65	0.7	0.75	0.8
0.85	0.9	0.95	1.0

increased to accommodate more training sets. In the model, loss function is 'mse' and 'ReLU' is the activation function selected for training. In every iteration, a new IW is predicted. Using eqs. (10) and (7), the projected IW is used to move the swarm. When the stopping criterion is met, the operation is stopped. The performance of CNNIWPSO is compared with CIWPSO, RIWPSO, and LDIWPSO in terms of MAE and SD for varying swarm sizes and dimensions. The BMF used for evaluating the performance of IWPSOs discussed are listed in Table 2.

Figure 2 depicts the flowchart for IWPSO using any desired IW computation model. This flowchart is generic. By choosing the appropriate IW computation model, IWPSO becomes model-specific IWPSO. For example, if the IWPSO uses the LDIW model for computing IW, the resulting PSO is LDIWPSO. Similarly, CIWPSO, RIWPSO, and CNNIWPSO are obtained.

The algorithm for CNNIWPSO is described below.

Algorithm 1: CNNIWPSO

Input: Swarm size, dimensions, iterations, error, fitness function boundaries

Output: Fitness value, convergence time etc.,

Step 1: Initialization

For each Particle, P_i, in the population

    Initialize px_i with uniform distribution

    Initialize pv_i randomly.

Evaluate the objective function with px_i and assign the value to fitness[i].

Initialize $pbest_i$ with a copy of px_i.

Initialize $pbest\_fitness_i$ with a copy of $fitness_i$.

Initialize pgbest with the index of the particle with the least fitness.

Build and Train CNN for Inertia Weight Prediction

Predict the new Inertia Weight.

Step 2: Swarm Movement

Repeat until the stopping criterion is reached

    For each Particle, P_i:

        Update pv_i and px_i

        Evaluate $fitness_i$

        If $fitness_i < pbest\_fitness_i$ then

            $pbest_i = px_i$

            $pbest\_fitness_i = fitness_i$

    Update pgbest by the particle with current least fitness among the population

    Predict the new Inertia Weight using trained CNN model

Table 2 Benchmark functions

No.	Benchmark Function Name	Interval	Best fitness value at
f1	Ackley	[−32, +32]	f(0)=0
f2	Alpine	[0, 10]	f(0)=0
f3	Rastrigin	[−5.12, +5.12]	f(0)=0
f4	Rosenbrock	[−5, 10]	f(1)=0
f5	Sphere	[−5.12, +5.12]	f(0)=0

4 Proposed LSTM-based IWPSO

4.1 Recurrent neural network

Recurrent Neural Networks (RNN) are discrete time-dynamic systems that interact with input vector sequences [1, 8, 24]. Traditionally, RNNs spread knowledge forward in time, forming forecasts based solely on past and current inputs. Figure 3 depicts the primary Recurrent Neural Network. Figure 4 shows the memory of an LSTM with dependencies.

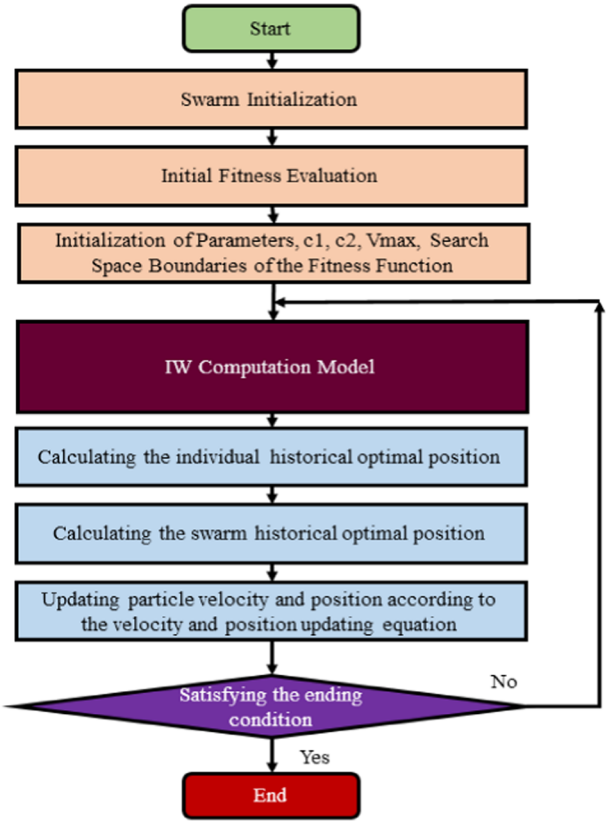


Fig. 2 Flowchart for generic IWPSO with IW computation model

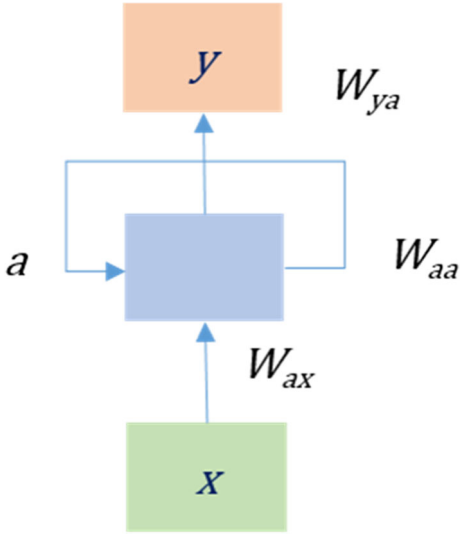


Fig. 3 A Recurrent Neural Network

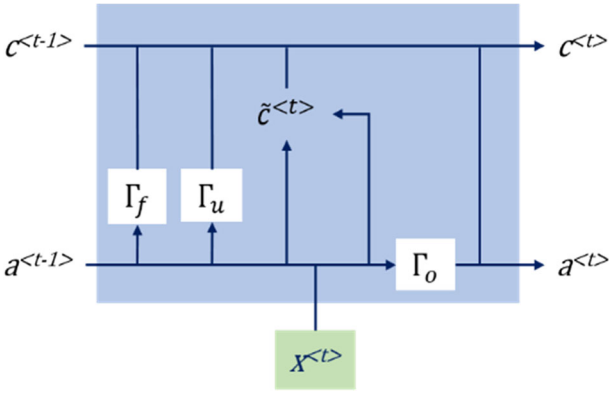


Fig. 4 Memory cell of an LSTM showing dependencies

For the traditional RNN, for each time step *t*, the output is computed using eq. (11), and the activation function $a^{<t>}$ is computed using eq. (12).

$$y^{<t>} = h\left(W_{ya}a^{<t>} + b_y\right). \tag{11}$$

$a^{<t>} = g(W_{aa}a^{<t-1>} + W_{ax}x^{<t-1>} + b_a)$, (12).

where t represents time, $y^{<t>}$ is the predicted value, W_{ya}, W_{aa}, W_{ax}, b_y, and b_a are the coefficients, and h and g are the activation functions. Generally, activation functions sigmoid, tanh, Leaky ReLU, ELU, PReLU, and ReLU are used.

4.2 Long short-term memory

Long-Term Short Memory is a specific type of RNN architecture that can learn dependency for the long term. Hochreiter and Schmidhuber [4, 9, 27] introduced efficient and effective, gradient-based LSTM. Figure 4 depicts the dependencies of the memory cell of an LSTM representing dependencies.

To deal with the vanishing gradient problem, The LSTM has the power to delete or add information to a cell state that is carefully controlled by mechanisms called gates [9]. LSTM uses three gates called update gate (Γ_u), forget gate (Γ_f) and output gate (Γ_o). The computation of $\tilde{c}^{<t>}$, $c^{<t>}$, $a^{<t>}$, Γ_u, Γ_f, Γ_o are shown through eq. (13) – eq. (18).

$$\tilde{c}^{<t>} = \tanh\left(w_c\left[a^{<t-1>}, x^{<t>}\right] + b_c\right) \tag{13}$$

$$\Gamma_u = \sigma\left(w_u\left[a^{<t-1>}, x^{<t>}\right] + b_u.\right. \tag{14}$$

$$\Gamma_f = \sigma\left(w_f\left[a^{<t-1>}, x^{<t>}\right] + b_f.\right. \tag{15}$$

$$\Gamma_o = \sigma\left(w_o\left[a^{<t-1>}, x^{<t>}\right] + b_o.\right. \tag{16}$$

$$c^{<t>} = \Gamma_u * \tilde{c}^{<t>} + \Gamma_f * c^{<t-1>}. \tag{17}$$

$$a^{<t>} = \Gamma_o * \tanh c^{<t>}. \tag{18}$$

4.3 LSTMIWPSO details

In LSTMIWPSO, the new inertia weight is computed using a univariate LSTM [11, 12, 27, 32]. Initially, an LSTM is built. Then, the LSTM model is built and trained with different inertia weights from 0.05 to 1.00. The sample set of IW with interval 0.05 is given as iw = [0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.55, 0.6, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95, 1.0]. The training and test set are shown in Table 1. By reducing the interval, the length of the IW set is increased to accommodate more training sets. In the model, the loss function is 'mse' and 'ReLU' is the activation function selected for training. In every iteration, a new inertia weight is predicted. Using eqs. (10) and (7), the projected IW is used to move the swarm. When the stopping criterion is met, the operation is stopped. The performance of LSTMIWPSO is compared with CIWPSO, RIWPSO, and LDIWPSO in terms of MAE and SD for varying swarm sizes and dimensions.

The flowchart for an implementation of LSTMIWPSO is obtained from Fig. 2, including the LSTMIW model. In addition, the LSTMIWPSO algorithm is described in the following.

Algorithm 2: LSTMIWPSO

Input: Swarm Size, dimensions, iterations, error, fitness function boundaries

Output: Fitness value, convergence time etc.,

Step 1: Initialization

For each Particle, P_i, in the population

    Initialize px_i with uniform distribution

    Initialize pv_i randomly.

Evaluate the objective function with px_i and assign the value to fitness[i].

Initialize $pbest_i$ with a copy of px_i.

Initialize $pbest\_fitness_i$ with a copy of $fitness_i$.

Initialize pgbest with the index of the particle with the least fitness.

Build and Train LSTM for Inertia Weight Prediction

Predict the new Inertia Weight.

Step 2: Swarm Movement

Repeat until the stopping criterion is reached

    For each Particle, P_i:

        Update pv_i and px_i

        Evaluate $fitness_i$

        If $fitness_i < pbest\_fitness_i$ then

            $pbest_i = px_i$

            $pbest\_fitness_i = fitness_i$

    Update pgbest by the particle with current least fitness among the population

    Predict the new Inertia Weight using trained LSTM model

5 Results and discussions

Experiments performed with various IWPSOs like CIWPSO, RIWPSO, LDIWPSO, CNNIWPSO, and LSTMIWPSO on five optimization test problems or BMF has shown in Table 2. The experimental procedure is shown in Table 3. Swarm sizes with 50, 75, and 100 particles with different dimensions, 10, 15, and 25, considered for conducting experiments.

Table 3 Experimental setting for CIWPSO, RIWPSO, LDIWPSO, CNNIWPSO and LSTMIWPSO

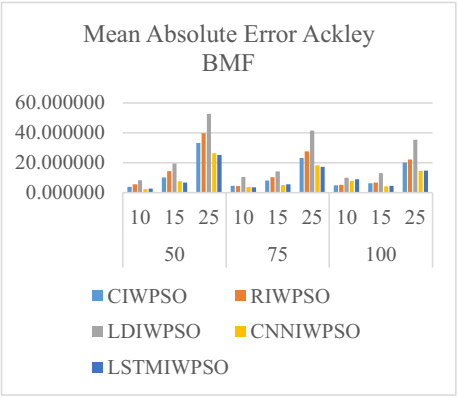
Swarm Size	Dimension	Inertia Weight(s)	Benchmark function	No of Iterations	No. of Simulations
50, 75, 100	10, 15, 25	CIW, RIW, LDIW, CNNIW and LSTMIW	Ackley, Alpine, Rastrigin, Rosenbrock, Sphere	1000	15

Five benchmark functions are used as objective functions to evaluate the performance of CIWPSO, RIWPSO, LDIWPSO, CNNIWPSO, and LSTMIWPSO in terms of MAE and SD. In total, 225 experiments (3 Swarm Sizes X 3 dimensions X 5 IWPSOs X 5 BMFs) are conducted to collect the results in terms of MAE and SD. Each experiment is executed for a maximum of 1000 iterations. In total, 15 simulations ran to reduce the effect of the randomness for each experiment. The performance of CNNIWPSO and LSTMIWPSO is compared with existing CIWPSO, RIWPSO, and LDIWPSO in terms of MAE and SD. The results are depicted as graphs, shown in the Figs. 5 and 6.

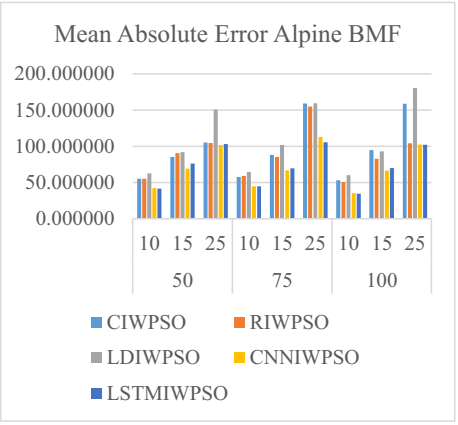
From Fig. 5(a), it is clear that CNNIWPSO is found to be superior in terms of MAE, for the benchmark function Ackley, for the swarm sizes 50 (Dim: 10), 75 (Dim: 15) and 100 (Dim: 15, 25). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 15, 25), 75 (Dim: 10, 25). CIWPSO is superior for the swarm size 100 (Dim: 10). From Fig. 5(b), it is evident that CNNIWPSO is found to be superior in terms of MAE, for the benchmark function Alpine, for the swarm sizes 50 (Dim: 15, 25), 75 (Dim: 15), and 100 (Dim: 15). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 10), 75 (Dim: 10, 25) and 100 (Dim: 10, 25). From Fig. 5(c), it is apparent that CNNIWPSO is found to be superior in terms of MAE for the benchmark function Rastrigin, for the swarm sizes 50 (Dim: 25) and 75 (Dim: 15, 25). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 10, 15), and 100 (Dim: 15, 25). RIWPSO is superior for the swarm sizes 75 (Dim: 10) and 100 (Dim: 10). From Fig. 5(d), it is clear that CNNIWPSO is found to be superior in terms of MAE, for the benchmark function Rosenbrock, for the swarm sizes 50 (Dim: 10, 15, 25), and 100 (Dim: 15). LSTMIWPSO is superior for the swarm sizes 75 (Dim: 15, 25) and 100 (Dim: 25). RIWPSO is superior for the swarm sizes 75 (Dim: 10) and 100 (Dim: 10). From Fig. 5(e), it is marked that CNNIWPSO is found to be superior in terms of MAE, for the benchmark function Rosenbrock, for the swarm sizes 50 (Dim: 10, 15, 25), and 100 (Dim: 15). LSTMIWPSO is superior for the swarm sizes 75 (Dim: 15, 25) and 100 (Dim: 25). RIWPSO is superior for the swarm sizes 75 (Dim: 10) and 100 (Dim: 10).

From Fig. 6(a), it is observed that CNNIWPSO is found to be superior in terms of SD, for the benchmark function Ackley, for the swarm sizes 50 (Dim: 10), 75 (Dim: 10, 15), and 100 (Dim: 15). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 15, 25), 75 (Dim: 25) and 100 (Dim: 25). CIWPSO is superior for the swarm size 100 (Dim: 10). From Fig. 6(b), it is noticeable that CNNIWPSO is found to be superior in terms of SD, for the benchmark function Alpine, for the swarm sizes 50 (Dim: 10, 15, 25), 75 (Dim: 15) and 100 (Dim: 10, 15). LSTMIWPSO is superior for the swarm sizes 75 (Dim: 10, 25), 100 (Dim: 25). From Fig. 6(c), it is obvious that CNNIWPSO is found to be superior in terms of SD for the benchmark function Rastrigin, for the swarm sizes 50 (Dim: 10, 15) and 75 (Dim: 25). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 25), 75 (Dim: 10, 25), 100 (Dim: 15, 25). RIWPSO is superior for the swarm size 100 (Dim: 10). From Fig. 6(d), it is marked that CNNIWPSO is found to be superior in terms of SD, for the benchmark function Rosenbrock, for the swarm sizes 50 (Dim: 10, 25), 75 (Dim: 25), and 100 (Dim: 15). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 15), 75 (Dim: 10, 15), 100 (Dim: 25). RIWPSO is superior for the swarm size 100 (Dim: 10). From Fig. 6(e), it is observed that CNNIWPSO is found to be superior in terms of SD for the benchmark function Sphere for the swarm sizes 50 (Dim: 10, 15) and 75 (Dim: 15). LSTMIWPSO is superior for the swarm sizes 50 (Dim: 25), 75 (Dim: 25), 100 (Dim: 15, 25). RIWPSO is superior for the swarm sizes 75 (Dim: 10) and 100 (Dim: 10).

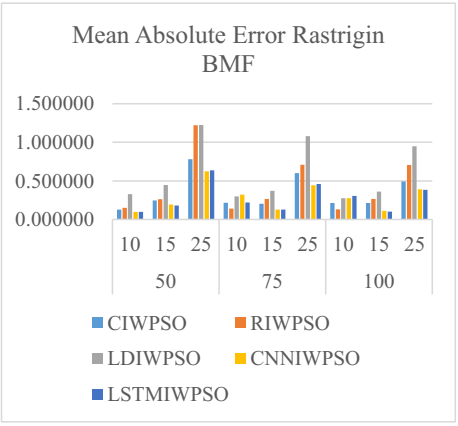
The performance of the models is acquired and tabulated in Tables 4 and 5—the percentage of models' performance from the perspective of MAE and SD are depicted in Figs. 7 and 8. It



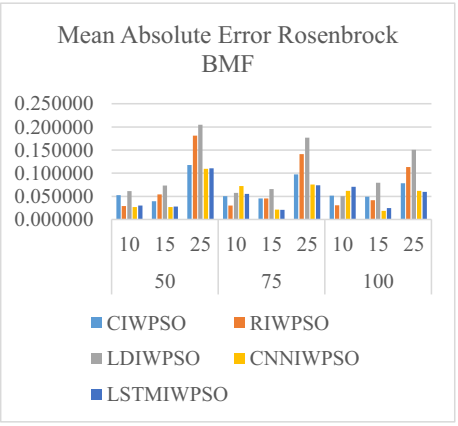
(a) MAE for Ackley BMF



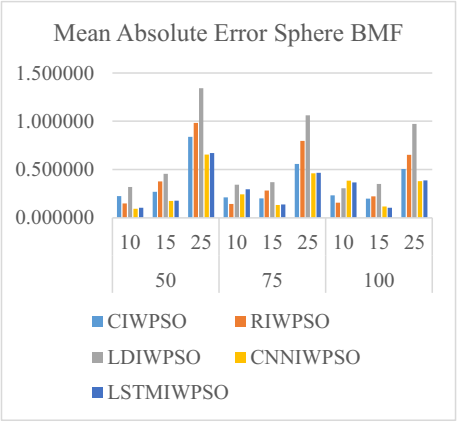
(b) MAE for Alpine BMF



(c) MAE for Rastrigin BMF

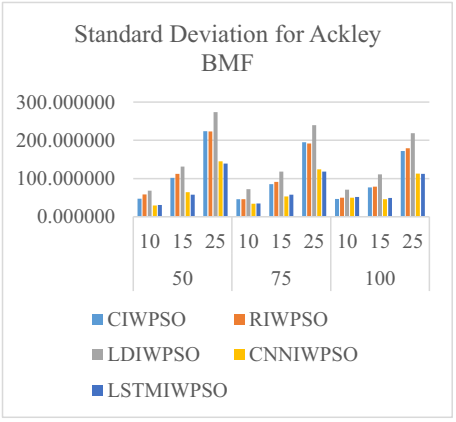


(d) MAE for Rosenbrock BMF

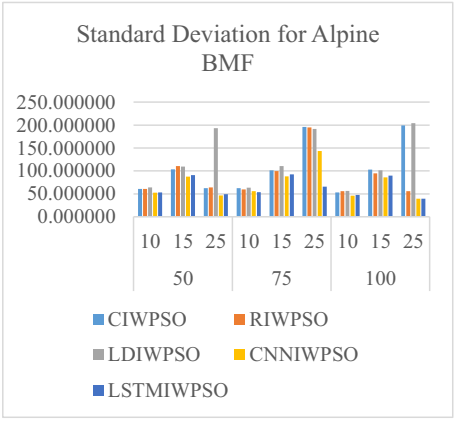


(e) MAE for Sphere BMF

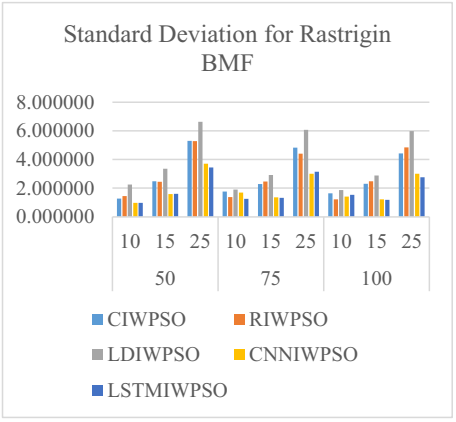
Fig. 5 MAE obtained for using Ackley, alpine, Rastrigin, Rosenbrock and Sphere BMF. (a) MAE for Ackley BMF. (b) MAE for Alpine BMF. (c) MAE for Rastrigin BMF. (d) MAE for Rosenbrock BMF. (e) MAE for Sphere BMF



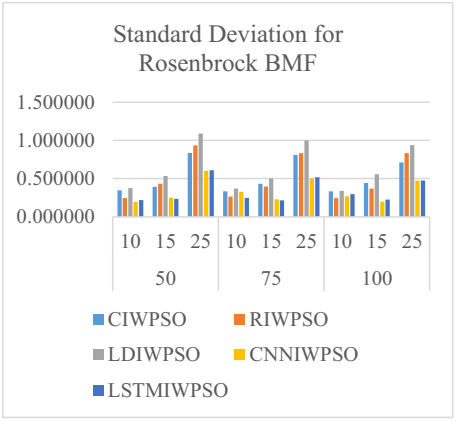
(a) SD for Ackley BMF



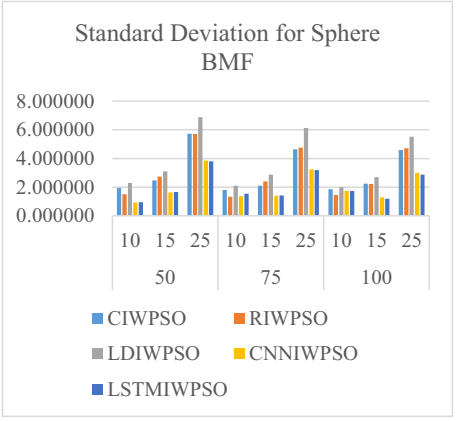
(b) SD for Alpine BMF



(c) SD for Rastrigin BMF



(d) SD for Rosenbrock BMF



(e) MAE for Sphere BMF

Fig. 6 SD obtained for using Ackley, alpine, Rastrigin, Rosenbrock and Sphere BMF. (a) SD for Ackley BMF. (b) SD for Alpine BMF. (c) SD for Rastrigin BMF. (d) SD for Rosenbrock BMF. (e) MAE for Sphere BMF

Table 4 Performance comparison concerning Mean Absolute Error (MAE)

BMF	Swarm Size and Dimensions								
	50			75			100		
	10	15	25	10	15	25	10	15	25
f1	A	B	B	B	A	B	E	A	A
f2	B	A	A	B	A	B	B	A	B
f3	B	B	A	D	A	A	D	B	B
f4	A	A	A	D	B	B	D	A	B
f5	A	A	A	D	A	A	D	B	A

A – CNNIWPSO, B – LSTMIWPSO, C – LDIWPSO, D – RIWPSO, E – CIWPSO.

Table 5 Performance comparison concerning SD

BMF	Swarm Size and Dimensions								
	50			75			100		
	10	15	25	10	15	25	10	15	25
f1	A	B	B	A	A	B	E	A	B
f2	A	A	A	B	A	B	A	A	B
f3	A	A	B	B	B	A	D	B	B
f4	A	B	A	B	B	A	D	A	B
f5	A	A	B	D	A	B	D	B	B

A – CNNIWPSO, B – LSTMIWPSO, C – LDIWPSO, D – RIWPSO, E – CIWPSO.

is observed from Table 4 and Fig. 7 that CNNIWPSO has better results in terms of MAE with 46.7% of experiments conducted. LSTMIWPSO stands second with 37.8%, whereas RIWPSO and CIWPSO acquired 13.3% and 2.2%, respectively, in the overall performance.

It is noticeable from Table 5 and Fig. 8 that CNNIWPSO and LSTMIWPSO have better results in terms of SD with 44.4% of experiments conducted. In contrast, RIWPSO and CIWPSO acquired 8.9% and 2.2%, respectively, in the overall performance.

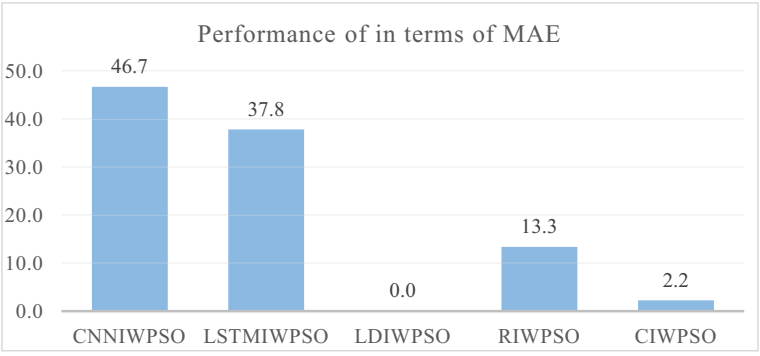


Fig. 7 Comparison of IWPSOs performance in terms of MAE

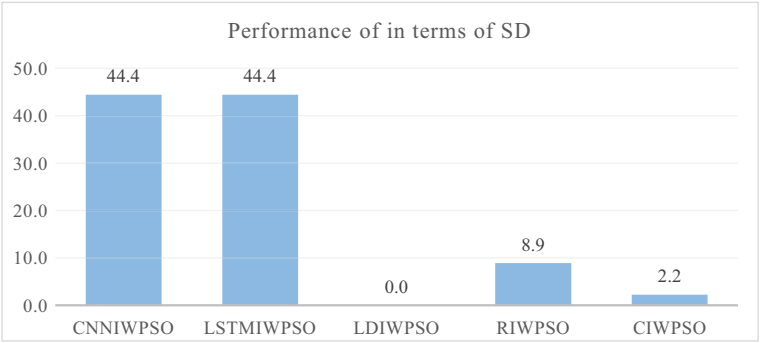


Fig. 8 Comparison of IWPSOs performance in terms of SD

6 Conclusion and future work

A univariate CNN and LSTM models are built and trained using IW from 0.05 to 1.0. These models predict IW, for IWPSO, in every iteration until a stopping criterion is reached. The performance of CNNIWPSO and LSTMIWPSO are compared with existing CIWPSO, RIWPSO, LDIWPSO in terms of MAE and SD. The experiments are conducted to assess the performance of the said models with swarm sizes 50, 75, and 100 with dimensions 10, 15, and 25 for each swarm size. The model performance is assessed with the help of standard benchmarking features like Ackley, Alpine, Rastrigin, Rosenbrock, and Sphere in terms of MAE and SD.

The experimental results show that CNNIWPSO has better results than LSTMIWPSO in terms of MAE. When swarm sizes are small, CNNIWPSO generated better MAE, whereas as swarm size increases, LSTMIWPSO performance is superior over CNNIWPSO. RIWPSO gave better MAE for the swarm size 100 and dimension 10. In terms of SD, CNNIWPSO and LSTMIWPSO performance is equally recorded. In modest cases, RIWPSO and CIWPSO have shown their presence.

In the future, the performance of CNNIWPSO and LSTMIWPSO is compared with other existing PSOs and other metaheuristic algorithms, with more complex BMFs and well-defined optimization problems. The experiments in terms of convergence time may be conducted. Further, the length of the IW set, the parameters of CNN, LSTM, and IWPSO, will be adjusted to improve the performance of CNNIWPSO and LSTMIWPSO.

Funding The author declares that they do not have any funding or grant for the manuscript.

Declarations

Conflict of interest The authors declare that they do not have any conflict of interests that influence the work reported in this paper.

Ethical approval No animals were involved in this study. All applicable international, national, and/or institutional guidelines for the care and use of animals were followed.

References

1. Amidi A, Amidi S. Recurrent Neural Network Cheatsheet/ https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks#architecture, Last Accessed on Apr 07 2021.
2. Bansal JC, Singh PK, Saraswat M, Verma A, Jadon SS, Abraham A (2011) Inertia weight strategies in particle swarm optimization, Third World Congr. on Nature and Biologically Inspired Comput, Salamanca, 633–640
3. Bonyadi MR, Michalewicz Z (2017) Impacts of coefficients on movement patterns in the particle swarm optimization algorithm. IEEE Trans Evol Comput 21(3):378–390
4. Chowdhury, Subrata, G, Ramya, Kolla, Bhanu, Donepudi, Babitha, Ismail, Mohammed, (2020), "Automated road safety surveillance system using hybrid CNN-LSTM approach," Int J of Adv Trends in Comput Sci and Eng, Vol. 9, pp. 1767–1773.
5. Cui Z, Shi Z (2009) Boid particle swarm optimisation. Int J Innov Comput Appl 2(2):77–85
6. Engelbrecht AP (2007) Computational Intelligence: An Introduction, John Wiley and Sons, 2007, ch. 16, 289–358.
7. Freitas D, Lopes LG, Morgado-Dias F (2020) Particle Swarm Optimisation: A Historical Review Up to the Current Developments. Entropy 22(3):1–36 Article No 362
8. Hammer B (2000) On the approximation capability of recurrent neural networks. Neuro Comput 31(1–4): 107–123
9. Hochreiter S, Schmidhuber J (1997) Long Short-term Memory. Neural Comput 9(8):1735–1780
10. Imambi S, Prakash KB, Kanagachidambaresan GR (2021) PyTorch. In: Prakash K.B., Kanagachidambaresan G.R. (eds) programming with TensorFlow. EAI/springer innovations in communication and computing. Springer, Cham., pp. 87-104, DOI: https://doi.org/10.1007/978-3-030-57077-4_10.
11. Jha AK, Ruwali A, Prakash KB, Kanagachidambaresan GR (2021) Tensorflow basics. In: Prakash K.B., Kanagachidambaresan G.R. (eds) programming with TensorFlow. EAI/springer innovations in communication and computing. Springer, Cham., pp. 5-15, https://doi.org/10.1007/978-3-030-57077-4_2
12. Kanagachidambaresan G.R., Prakash K.B., Mahima V. (2021) Programming tensor flow with single board computers. In: Prakash K.B., Kanagachidambaresan G.R. (eds) programming with TensorFlow. EAI/springer innovations in communication and computing. Springer, Cham., pp. 145-157, https://doi.org/10.1007/978-3-030-57077-4_12
13. Kennedy J, Eberhart R (1995) Particle Swarm Optimization, in Proc. of IEEE Int Conf on Neural Networks, pp. 1942–1948.
14. Kumar A, Singh BK, Patro BDK (2016) Particle swarm optimization: a study of variants and their applications. Int J Comput Appl 135(5):24–30
15. Kushwaha N, Pant M (2019) Modified particle swarm optimization for multimodal functions and its application. Multimed Tools Appl 78:23917–23947
16. Lin YH, Hu YC (2018) Electrical energy management based on a hybrid artificial neural network-particle swarm optimization-integrated two-stage non-intrusive load monitoring process in smart homes. Processes 6(12):236
17. Liu T, Yin S (2017) An improved particle swarm optimization algorithm used for BP neural network and multimedia course-ware evaluation. Multimed. Tools Appl 76, 11961–11974.
18. Marini F, Walczak B (2015) Particle swarm optimization (PSO). A tutorial. Chemom Intell Lab Syst 149(Part B):153–165
19. Oldewage ET, Engelbrecht AP, Cleghorn CW (2017) The merits of velocity clamping particle swarm optimization in high dimensional spaces. IEEE Symposium Series on Computational Intell. (SSCI), Honolulu, HI, pp. 1–8
20. Pan F, Chen D, Lu L (2020) Improved PSO based clustering fusion algorithm for multimedia image data projection. Multimed Tools Appl 79:9509–9522
21. Parsopoulos KE, Vrahatis MN (2002) Initialising the particle swarm optimizer using the nonlinear simplex method, Advances in Intell Syst, Fuzzy Syst, Evol Comput WSEAS Press, pp. 216–221
22. Piotrowski AP, Napiorkowski JJ, Piotrowska AE (2020) Population size in particle swarm optimization. Swarm and Evol Comput 58, Article No. 100718, pages 18, https://doi.org/10.1016/j.swevo.2020.100718.
23. Poli R (2008) Analysis of the Publications on the Applications of Particle Swarm Optimisation. J of Artif Evolution and Appl 2008, Article No. 685175., 10 pages, https://doi.org/10.1155/2008/685175
24. Prakash KB (2020) Accurate hand gesture recognition using CNN and RNN approaches. Int J of Adv Trends in Comput Sci and Eng 9:3216–3222
25. Prakash KB (2020) Predicting CryptoCurrency prices using machine learning and deep learning techniques. Int J of Adv Trends in Comput Sci and Eng 9:6603–6608
26. Reynolds CW (1987) Flocks, herds and schools: A distributed behavioural model, in Proc. of the 14th ACM Annu. Conf. on Compu Graph and Interact Techn (SIGGRAPH '87), pp. 25–34. 10.1145/37401.37406.

27. Ruwali A, Kumar AJS, Prakash KB, Sivavaraprasad G, Ratnam DV (2021) Implementation of hybrid deep learning model (LSTM-CNN) for ionospheric TEC forecasting using GPS data. IEEE Geosci Remote Sens Lett 18(6):1004–1008
28. Serani A, Leotardi C, Iemma U, Campana E, Fasano G, Diez M (2016) Parameter selection in synchronous and asynchronous deterministic particle swarm optimization for ship hydrodynamics problems. Appl Soft Comput 49:313–334
29. Shahzad F, Baig AR, Masood S, Kamran M, Naveed N (2009) Opposition-based particle swarm optimization with velocity clamping (OVCPSO). In: Yu W, Sanchez EN (eds) Advances in Intell and Soft Comput 116, 339–348.
30. Shi Y, Eberhart R (2009) A modified particle swarm optimizer. IEEE World Congr on Computational Intell, 66–69
31. Tejasri K, Srinivas M, Prakash KB, Pavan Kumar T (2020) Heart disease diagnosis using ANN, RNN and CNN. Int J of Adv Sci and Tech 29:2232–2239
32. Vamsidhar E, Kanagachidambaresan GR, Prakash KB (2021) Application of machine learning and deep learning. In: Prakash K.B., Kanagachidambaresan G.R. (eds) programming with TensorFlow. EAI/springer innovations in communication and computing. Springer, Cham. 63-74, https://doi.org/10.1007/978-3-030-57077-4_8
33. van den Bergh F (2006) An Analysis of Particle Swarm Optimizers, PhD thesis, Department of Computer Science., University of Pretoria, Pretoria, South Africa.
34. Yellapragada, Bharadwaj, Rajaram P, Sriram VP, Sengan S, Kolla B (2020) Effective Handwritten Digit Recognition using Deep Convolution Neural Network. Int J of Adv Trends in Comput Sci and Eng 9:1335–1339
35. Zhang Y, Wang S, Ji G (2015) A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications. Mathematical Problems in Engineering 2015, Article No. 931256, 38 pages, https://doi.org/10.1155/2015/931256.
36. Zhu H, Zhuang Z, Zhou J, Zhang F, Wang X, Wu Y (2017) Segmentation of liver cyst in ultrasound image based on adaptive threshold algorithm and particle swarm optimization. Multimed Tools Appl 76:8951–8968

Y.V.R. Naga Pawan received his Ph.D degree. in Computer Science and Engineering from Koneru Lakshmaiah Education Foundation, Vijayawada, A.P., INDIA. He is working as Associate Professor in Anurag Engineering College, Ananthagiri, Telangana, INDIA. The areas of interest are Databases, Network Security, Machine Learning, Deep Learning, and Data Visualization.

Kolla Bhanu Prakash is Ph.D. from Satyabhama University and working as a Professor in the Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Vijayawada, A.P., INDIA. The areas of interest are Machine Learning, Knowledge Engineering.

Subrata Chowdhury received his Ph.D. degree in Computer Science from the Computer Science and Application Department from Vels University. He is working as a professor in the SVCET Engineering College, A.P, INDIA. The areas of interest are Machine Learning, Cloud Computing, and IoT.

Yu-Chen Hu received his Ph.D. Degree in Computer Science and Information Engineering from the Department of Computer Science and Information Engineering, National Chung Cheng University, Chiayi, Taiwan. Currently, Dr. Hu is a professor in the Department of Computer Science and Information Management, Providence University, Taiwan. His research interests include image and signal processing, data compression, information hiding, information security, computer network, and machine learning.